

Backpropagation through monoidal folds and scans.

Amaru Cuba Gyllenstein

...

Abstract

Recent advances in memory-efficient neural network layers, such as FlashAttention, often appear as specialized engineering solutions but share a common mathematical structure. We show that many of these kernels can be understood as folds over monoids, with especially strong consequences in the commutative case. This perspective unifies MapReduce-style computation with modern deep learning optimizations. Building on this, we introduce the Local Gradient Property, which provides a sufficient condition under which gradients of monoidal folds can be computed locally from the final monoidal product and individual factors, enabling efficient backward passes. We extend this to scans over sequences, which enables efficient, parallel, backward passes that can reduce the object scanned in the backward pass, improving span and communication in nontrivial cases. We also extend it to mixed-axis folds over scans, where the scan produces significant internal state that is then folded over; regaining some of the memory efficiency of folds for scans.

1 Introduction

Modern deep learning optimizations increasingly rely on custom, hardware-aware kernels to bypass the memory bandwidth bottlenecks of standard automatic differentiation (AD). Parallel reductions and sequence-wide scans—found in attention mechanisms, state-space models (SSMs), and large-vocabulary classification layers—are typically optimized through specialized, hand-crafted implementations. Although these kernels are often treated as isolated engineering achievements, they share a common mathematical foundation: they can be formalized as folds or scans over monoidal algebraic structures.

While the forward pass of monoidal reductions or scans are readily parallelized using standard primitives such as MapReduce or Blelloch scans, parallelizing their backward pass under strict memory constraints remains a key challenge. Standard AD engines typically force the materialization of large intermediate variables or propagate high-dimensional Jacobians across the parallel execution graph. To address this, we present a unified theoretical framework that identifies sufficient algebraic and algorithmic conditions under which these

backward passes can be computed efficiently, parallelly, and with minimal memory footprint. Specifically, we present the following contributions:

- **The Local Gradient Property (LGP):** We introduce the LGP as a sufficient algebraic condition for parallel monoidal folds (reductions). If the derivative of a monoidal binary operation can be computed using its output and individual input factors, the global gradient can be evaluated using only local quantities and the final result. In both the general and commutative cases, this property allows the backward pass of a reduction to be executed in parallel with minimal memory storage, at the cost of recomputation.
- **The Finalize-Embed Factorization:** To extend these benefits to parallel scans, where gradients must propagate across multiple intermediate prefix outputs, we introduce the *finalize-embed factorization*. This factors the LGP-derivative through an auxiliary, gradient accumulation semigroup $(A, \oplus)$. Instead of performing the backward pass as a scan over compositions of Jacobian matrices—which is typical in general-case automatic differentiation—the factorization reduces the backward pass to a parallel suffix scan over the semigroup $(A, \oplus)$ (typically, additive). Beyond scans, we show that this factorization simplifies block-wise tiled folds by acting as an efficient boundary interface for local gradient accumulation.
- **Execution Carriers:** Recognizing that abstract algebraic monoids are often numerically unstable in finite-precision floating-point arithmetic (such as the exponentiation within online softmax), we introduce the concept of *Execution Carriers*. These are monoid homomorphisms that map numerically stable, hardware-aware execution representations to their underlying semantic targets. This mechanism preserves algebraic differentiability while maintaining numerical stability, generalizing the fused max and exp2-trick used in FlashAttention Dao et al. (2022); Dao (2023).
- **Parallel Scan-Fold Fusion:** We extend these concepts to a fused *Scan-Fold* primitive, which integrates sequential scans with state-wise commutative folds. By concurrently applying the LGP to both axes, the Scan-Fold primitive achieves the parallel speedups of scans while retaining the space efficiency of folds, processing high-dimensional intermediate states without fully materializing them in either the forward or backward pass.
- **Transport and Instances:** We establish transport theorems showing how the LGP and the finalize-embed factorization of a monoid T are inherited by any monoid S that admits an injective, smooth homomorphism into T . Notably, and trivially, $GL_N(\mathbb{R})$ and $(\mathbb{R}^N, +)$ satisfy the LGP and have finalize-embed factorizations. We then apply this framework to systematically derive memory-efficient parallel passes for sequence models, attention mechanisms, cross-entropy losses, and information-theoretic measures. We also show that the formalism extends beyond those smooth

examples by showing that $(\mathbb{R}, \max)$ satisfies the LGP and has a finalized-embed factorization.

1.1 Notation

Types and Products: We write $a : A$ to denote that a is of type A . Named product types are written as $\{a : A, b : B\}$, with corresponding instantiated terms written as $\{a : v, b : w\}$.

Functions and Application: Function arrows associate to the right: $A \rightarrow B \rightarrow C \equiv A \rightarrow (B \rightarrow C)$. Standard parentheses $f(x)$ denote function application on primal parameters or state. Conversely, square brackets $f[v]$ are reserved for operations acting on dual objects (gradients, cotangents, or accumulations). A parameterized map acting on a dual object v is written as $f(x)[v]$.

Composition and Multiplication: Symbol $\circ$ denotes function composition: $(f \circ g)(x) = f(g(x))$. Matrix multiplication is denoted using standard juxtaposition (e.g., AB).

Derivatives and Pullbacks: Inspired by Elliott (2018), the derivative $\frac{da}{db} : A^* \rightarrow B^*$ denotes the pullback linear map (the transpose Jacobian) that transforms cotangents (gradients) of $a : A$ into cotangents of $b : B$. Accordingly, the chain rule is written as the composition of these linear maps: $\frac{dc}{da} = \frac{db}{da} \circ \frac{dc}{db}$.

2 Related work

Memory-Efficient Kernels. To mitigate the memory and bandwidth bottlenecks of backpropagation, conventional approaches rely on gradient checkpointing (Chen et al., 2016), which trades recomputation for peak memory reduction based on optimal checkpointing schedules (Griewank and Walther, 2000). More recently, hardware-aware design has shifted towards custom fused kernels. FlashAttention (Dao et al., 2022; Dao, 2023) utilizes online softmax reductions to calculate attention without storing intermediate matrices. Similarly, modern libraries such as Liger Kernel (Hsu et al., 2024) and Cut Cross-Entropy (Wijmans et al., 2024) compute loss and gradients blockwise to avoid materializing giant logit layers in global memory. Our work formalizes these disparate engineering efforts under a unified algebraic perspective: they are commutative monoidal folds.

State Space Models. State Space models, likewise, employ associative scans, usually over composition of restricted affine maps, and develop memory and compute efficient kernels for these Yang et al. (2025); Guo et al. (2025); Gu and Dao (2024).

Differentiable Parallel Primitives. While abstract parallel paradigms like MapReduce (Dean and Ghemawat, 2008) have long been used to organize cluster-scale computation, modern frameworks such as DrJAX (Rush et al., 2024) bring fully differentiable MapReduce primitives to trace-based compilers. For general kernel operations, systems like KeOps (Charlier et al., 2021) optimize symbolic reductions on the fly. Most conceptually similar to our theoretical approach is the development of automatic differentiation (AD) in functional array compilers. Specifically, Schenck et al. (2022) and Bruun et al. (2023) derived source-to-source reverse-mode AD rules for parallel building blocks like reductions and scans.

Key Distinctions of Our Work. This paper differs from prior work on in three primary ways:

- **Algebraic Restructuring vs. Compiler-Level AD (Futhark, JAX):** Standard AD frameworks—whether local array compilers like Futhark (Schenck et al., 2022; Bruun et al., 2023) or distributed systems like DrJAX (Rush et al., 2024)—mechanically trace vector-Jacobian products (VJPs) over the forward computation graph. For scans over non-linear operators, this generic tracing forces the propagation of high-dimensional linear pullbacks (Jacobians) combined via function composition or matrix multiplication. Our framework uses the LGP to mathematically restructure the derivative itself. By factoring the derivative through an auxiliary additive partial semigroup $(A, \oplus)$, the backward scan is simplified to ordinary matrix or vector addition, moving the multiplicative steps into local, non-communicating phases. This reduces the computational span of the backward pass and minimizes the physical data volume that must be stored or transmitted across memory and communication boundaries.
- **Numerical Stabilization and Execution Carriers:** AD compilers are strictly evaluative; they differentiate the exact forward code written by the programmer. However, many key operators in machine learning (such as softmax or recurrent updates) are numerically unstable when written in their native semantic representation. Our framework introduces *Execution Carriers* to map hardware-efficient, numerically stabilized execution-space monoids to their semantic counterparts. This provides a structured mathematical template to manually derive stable, fused backward passes (such as the online max-trick in FlashAttention), addressing numerical representation details that typically fall outside the scope of standard automatic differentiation.
- **Parallel Scan-Fold Fusion:** Traditional AD treatments view scans and reductions as separate, sequentially composed primitives. This separation often requires compilers to fully materialize intermediate scan states in global memory to compute gradients. We introduce a parallel Scan-Fold primitive that fuses sequential scans with state-wise commutative folds. By concurrently leveraging the LGP for both operations, we bypass the

need to materialize the intermediate high-dimensional states in both the forward and backward passes.

3 The Local Gradient Property

First we introduce the Local Gradient Property (LGP), which comes in two versions:

- The General Local Gradient Property, for non-commutative monoids.
- The Commutative Local Gradient Property, for commutative monoids.

Both versions show that gradient computation for monoidal folds can be simplified and made memory-efficient: if the derivative of the binary operation can be computed based on its output and individual input, then a similar local gradient rule holds for monoidal products.

Proposition 3.1 (The Local Gradient Property). *Let T be a finite-dimensional vector space and $(T, \odot)$ a monoid with a derivative $\frac{da \odot b \odot c}{db}$ that can be expressed as a function (D) of $a \odot b \odot c$, $a \odot b$ and b :*

$$D : T \times T \times T \rightarrow T^* \rightarrow T^*$$

$$\frac{da \odot b \odot c}{db} = D(a \odot b \odot c, a \odot b, b) \quad (1)$$

Then for any global product $g = x_1 \odot x_{1+1} \odot \dots \odot x_N$, prefix $p = x_1 \odot x_{1+1} \odot \dots \odot x_n$, and local slice $l = x_m \odot x_{m+1} \odot \dots \odot x_n$, such that p is a prefix of g and l is a suffix of p (i.e. $1 \leq m \leq n \leq N$), the following holds:

$$\frac{dg}{dl} = D(g, p, l) \quad (2)$$

Proof. The result follows immediately from the monoid laws (See A.1) $\square$

Corollary 3.2 (The Commutative LGP). *In the commutative case, the prefix p can be ignored: if the derivative $\frac{da \odot b}{da}$ can be expressed as a function (D) of $a \odot b$ and a , then:*

$$\frac{dg}{dl} = D(g, l)$$

Proof. The result follows immediately from commutativity and the monoid laws (See A.2) $\square$

Corollary 3.3 (Binary form of the General LGP). *If $\frac{da \odot b}{da} = D_l(a \odot b, a)$, and $\frac{da \odot b}{db} = D_r(a \odot b, b)$, we can derive the ternary form used in proposition 3.1 using the chain rule:*

$$\begin{aligned}
\frac{d a \odot b \odot c}{d b} &= \frac{d a \odot b}{d b} \circ \frac{d a \odot b \odot c}{d a \odot b} \\
&= D_r(a \odot b, b) \circ D_l(a \odot b \odot c, a \odot b) \\
D(g, p, l) &= D_r(p, l) \circ D_l(g, p)
\end{aligned}$$

Note on the identity element While the LGP is stated for monoids, many common operations in deep learning (such as the binary operations in Attention or certain RNNs) are naturally semigroups where an identity element is not immediately present. However, any semigroup $(S, \odot)$ can be extended to a monoid by adjoining an identity element $\mathbf{1}$ such that $a \odot \mathbf{1} = \mathbf{1} \odot a = a$. In neural network layers, such an identity usually corresponds to a "null" initial state or a masked value whose derivatives need not be computed. In such cases, we can adjoin a purely formal identity element $\mathbf{1}$ and allow $D(g, l)$ (or $D(g, p, l)$) to remain undefined at $l = \mathbf{1}$, since these expressions are only used for $l \neq \mathbf{1}$.

Cancellability gives a special case of the LGP If there exists left and right-cancellation operations $\oslash, \otimes$, such that $a \oslash (a \odot b) = b$ and $(a \odot b) \otimes b = a$, then we get the LGP for free using standard Automatic Differentiation on reconstructed left and right hand side arguments: $g = lhs \odot l \odot rhs$, where $lhs = g \otimes p$, and $rhs = l \otimes p$. This is how memory efficient kernels are commonly characterized in SSM-litterature Gu and Dao (2024). However, as we shall see in Section 6.3, cancellability is a special, rather than only, case of the LGP.

Example: LogWeightedSum As an illustrative example, we introduce the LogWeightedSum monoid, which forms the basis for attention, cross-entropy, and other information theoretic measures:

$$\begin{aligned}
LWS &:= \{z : \mathbb{R}, \mu : \mathbb{R}^D\} \\
a \odot_{LWS} b &= \{z : \text{logaddexp}(a_z, b_z), a_\mu \cdot \exp(a_z - z) + b_\mu \cdot \exp(b_z - z)\}
\end{aligned}$$

The LWS monoid is commutative, and satisfies the commutative LGP:

$$\begin{aligned}
\frac{d a \odot b}{d a}[\Delta] &= \exp(a_z - (a \odot b)_z) \cdot \{z : \Delta_z + \langle \Delta_\mu, a_\mu - (a \odot b)_\mu \rangle, \mu : \Delta_\mu\} \\
&\Downarrow \\
D(g, l)[\Delta] &= \exp(l_z - g_z) \cdot \{z : \Delta_z + \langle \Delta_\mu, l_\mu - g_\mu \rangle, \mu : \Delta_\mu\}
\end{aligned}$$

4 The Finalize—Embed Factorization

The Local Gradient Property establishes that the derivative of a monoidal product with respect to any local factor can be computed from global, prefix, and

local quantities. Given a sequence $[x_1, x_2, \dots, x_N] : S^N$ of local factors, with prefix products $p_t = x_1 \odot x_{1+1} \odot \dots \odot x_t$, the LGP states that $\frac{d p_t}{d x_i} = D(p_t, p_i, x_i)$ for $t \geq i$. For folds, where we only run the backwards pass w.r.t. p_N , i.e. $\frac{d \ell}{d x_i} = D(p_N, p_i, x_i) \left[\frac{d \ell}{d p_N} \right]$, this poses no problem. However, if we apply the LGP for scans, where we need to account for $\frac{d \ell}{d p_t}$ for all $t \geq i$, we quickly encounter a problem:

$$\frac{d \ell}{d x_i} = \sum_{t \geq i} D(p_t, p_i, x_i) \left[\frac{d \ell}{d p_t} \right]$$

meaning that the gradient calculation for a local factor x_i need access to all future prefixes p_t and their gradients $\frac{d \ell}{d p_t}$. To address this limitation, we factor the local gradient function D_S through an auxiliary gradient accumulation (partial) semigroup $(A, \oplus)$ ¹. using functions `finalize` and `embed`:

$$\begin{aligned} \text{finalize}(p, l) &: A \rightarrow S^* && \text{(For general monoids)} \\ \text{finalize}(l) &: A \rightarrow S^* && \text{(For commutative monoids)} \\ \text{embed}(g) &: S^* \rightarrow A \end{aligned}$$

such that the following identities hold for $p_i = l_1 \odot l_{1+1} \odot \dots \odot l_i$:

$$\begin{aligned} D_S(g, p, l) &= \text{finalize}(p, l) \circ \text{embed}(g) \\ \sum_{t \geq i} D_S(p_t, p_i, l_i) \left[\frac{d \ell}{d p_t} \right] &= \text{finalize}(p_i, l_i) [\eta_i^\oplus] \\ \text{where } \eta_i^\oplus &= \bigoplus_{t \geq i} \text{embed}(p_t) \left[\frac{d \ell}{d p_t} \right] \end{aligned}$$

in the general case (and corresponding prefix-less formulation in the commutative case), resulting in a local gradient of the form

$$\frac{d \ell}{d x_i} = \text{finalize}(p_i, x_i) [\eta_i^\oplus]$$

from which it follows that local gradient calculation can be performed efficiently using suffix scans over embedded A s.

Example: Factorization of D_{LWS} The LGP-style gradient function for the LogWeightedSum monoid is:

$$D(g, l)[\Delta] = \exp(l_z - g_z) \cdot \{z : \Delta_z + \langle \Delta_\mu, l_\mu - g_\mu \rangle, \mu : \Delta_\mu\}$$

¹The accumulator A need not be a vector space. We require only a partially defined associative operation $\oplus$ sufficient for the suffix accumulations that arise in the backward pass. In additive and GL_N -transported cases, $\oplus$ is ordinary addition. For max-like scans, A may instead carry suffix attribution data, and $\oplus$ is defined only for compatible attribution objects.

By isolating terms dependent on g and factoring through an additive A , we get the following²:

$$\begin{aligned}\text{embed}(g)[\Delta] &= \exp(-g_z) \cdot \{z : \Delta_z - \langle \Delta_\mu, g_\mu \rangle, \mu : \Delta_\mu\} \\ \text{finalize}(l)[\eta] &= \exp(l_z) \cdot \{z : \eta_z + \langle \eta_\mu, l_\mu \rangle, \mu : \eta_\mu\}\end{aligned}$$

Which is not a numerically stable accumulator: $\exp(-g_z)$ and $\exp(l_z)$ can easily lead to under- or overflow. It does, however, illustrate an interesting duality: the gradient accumulator for a `LogWeightedSum` is a `LogWeightedSum`, giving the following alternative gradient accumulator:³

$$\begin{aligned}A &:= LWS(\{s : \mathbb{R}, v : \mathbb{R}^D\}) \\ \text{embed}(g)[\Delta] &= \{z : -g_z, s : \Delta_z - \langle \Delta_\mu, g_\mu \rangle, v : \Delta_\mu\} \\ \text{finalize}(l)[\eta] &= \exp(l_z + \eta_z) \cdot \{z : \eta_s + \langle \eta_v, l_\mu \rangle, \mu : \eta_v\}\end{aligned}$$

5 Execution carrier

In practice, performing a fold or scan using the semantic monoid $(S, \odot_S)$ can lead to numerical instability or performance issues. Instead, the fold is often performed in an execution carrier $(E, \odot_E)$, which can act as a numerically stable or hardware-efficient representation of S .

We define an execution carrier $(E, \odot_E)$ for a semantic monoid $(S, \odot_S)$ as a surjective monoid homomorphism $\Psi : (E, \odot_E) \rightarrow (S, \odot_S)$ with a right inverse $\Phi : S \rightarrow E$ such that

$$\begin{aligned}\Psi \circ \Phi &= \text{id}_S \\ \Psi(a \odot_E b) &= \Psi(a) \odot_S \Psi(b) \quad \forall a, b \in E\end{aligned}$$

From which it follows that $\odot_i l_i = \Psi(\odot_i \Phi(l_i))$. Crucially, this identity allows us to compute folds or scans in an execution carrier E , and utilize the LGP-backwards pass for reconstructed semantic primal states in S directly, computed via Ψ , rather than threading the gradient pass through the execution carrier. As such, execution carriers enables representational freedom in the computation of the reduction, detached from the gradient logic, which can enable more numerically stable or hardware efficient ways of computing the fold or scan over S .

²When applicable, multiplication and addition on products is applied componentwise, i.e. $a \cdot \{x : x, y : y\} = \{x : a \cdot x, y : a \cdot y\}$

³Here, $LWS(x)$ denotes a LWS -monoid with a weighted sum of type x . That is, for $LWS(\{s : \mathbb{R}, v : \mathbb{R}^D\})$

$$a \odot b = \left\{ \begin{array}{l} z : \text{logaddexp}(a_z, b_z) \\ s : \exp(a_z - z) \cdot a_s + \exp(b_z - z) \cdot b_s \\ v : \exp(a_z - z) \cdot a_v + \exp(b_z - z) \cdot b_v \end{array} \right\} \quad (3)$$

Example: Fused Max-Trick as an Execution Carrier for LWS When computing $\text{logaddexp}(a, b)$, a standard trick to improve numerical stability is to reformulate the expression so that the exponents are bounded:

$$\begin{aligned}\text{logaddexp}(a, b) &= c + \text{logaddexp}(a - c, b - c) \\ &= h + \text{logaddexp}(0, l - h) \quad \text{where } (h, l) = \begin{cases} (a, b) & a > b \\ (b, a) & a \leq b \end{cases} \\ &= h + \log(1 + \exp(l - h))\end{aligned}$$

Unrolling this, we can build an execution carrier for LogSumExp that tracks the maximum m and rescales the sum of exponents e accordingly:

$$\begin{aligned}E_{LSE} &:= \{m : \mathbb{R}, e : \mathbb{R}\} \\ a \odot_{E_{LSE}} b &= \{m : h_m, e : h_e + \exp(l_m - h_m) \cdot l_e\} \\ \text{where } (h, l) &= \begin{cases} (a, b) & a_m > b_m \\ (b, a) & a_m \leq b_m \end{cases} \\ \Phi(x) &= \{m : x, e : 1\} \\ \Psi(y) &= y_m + \log(y_e)\end{aligned}$$

For $x = \bigodot_i \Phi(z_i)$, we then have that $x_z = \max_i z_i$, and $x_e = \sum_i \exp(z_i - x_z)$. To extend this to the *LWS*-monoid, the value component μ can be treated in much the same way as e :

$$\begin{aligned}E_{LWS} &:= \{m : \mathbb{R}, e : \mathbb{R}, u : \mathbb{R}^D\} \\ a \odot_{E_{LWS}} b &= \{m : h_m, e : h_e + \alpha \cdot l_e, u : h_u + \alpha \cdot l_u\} \\ \text{where } (h, l) &= \begin{cases} (a, b) & a_m > b_m \\ (b, a) & a_m \leq b_m \end{cases} \\ \alpha &= \exp(l_m - h_m) \\ \Phi(x) &= \{m : x_z, e : 1, u : x_\mu\} \\ \Psi(y) &= \left\{ z : y_m + \log(y_e), \mu : \frac{y_u}{y_e} \right\}\end{aligned}$$

This algebraic structure corresponds to the online scaling and denominator tracking used in the forward pass of FlashAttention (Dao et al., 2022).

6 Transport

While we won't provide an exhaustive characterization of when operations satisfy the LGP and when they have a finalize-embed-factorization, we do identify one broad and important class for the general case, one for the commutative

case, and a general way to construct new LGP-satisfying operations from existing ones:

- If T satisfies the LGP, then any S that admits an injective, smooth homomorphism $S \hookrightarrow T$ with an inverse on its image satisfies the LGP. Furthermore, if T has a finalize-embed factorization through A , then so does S .
- Any monoid admitting an injective, smooth finite dimensional homomorphism into GL_N with an inverse on its image satisfies the General LGP and has a finalize-embed factorization.
- Any monoid admitting an injective, smooth finite dimensional homomorphism into $(\mathbb{R}^N, +)$ with an inverse on its image satisfies the Commutative LGP and has a finalize-embed factorization.

We begin by proving the former, from which the latter two follow. We also discuss LGP-style rules for non-cancellable monoids, namely windowing monoids and max under a chosen tie-counting subgradient convention.

Proposition 6.1 (Transport of the General LGP). *Let $(S, \odot)$ and $(T, \otimes)$ be monoids, and let*

$$\phi : S \rightarrow T$$

be an injective, smooth homomorphism $(S, \odot) \hookrightarrow (T, \otimes)$ with ϕ a smooth left-inverse on its image:

$$\psi : \phi(S) \rightarrow S \quad (\psi \circ \phi = \text{id})$$

If $(T, \otimes)$ satisfies the General Local Gradient Property with derivative rule D_T , then so does $(S, \odot)$ with

$$D_S(g, p, l) := \frac{d\phi(l)}{dl} \circ D_T(\phi(g), \phi(p), \phi(l)) \circ \frac{d\psi(\phi(g))}{d\phi(g)} \quad (4)$$

Proof. The result follows directly from the homomorphism properties and the chain rule (See appendix A.3). $\square$

Corollary 6.2 (Factorization transport). *With the same conditions as in proposition 6.1, if D_T can be factored into embed_T and finalize_T , then so can D_S , with:*

$$\begin{aligned} \text{embed}_S(g) &= \text{embed}_T(\phi(g)) \circ \frac{d\psi(\phi(g))}{d\phi(g)} \\ \text{finalize}_S(p, l) &= \frac{d\phi(l)}{dl} \circ \text{finalize}_T(\phi(p), \phi(l)) \end{aligned}$$

Proof. The result follows directly by substituting the factorization $D_T = \text{finalize}_T \circ \text{embed}_T$ in Proposition 6.1 and grouping the composition by associativity. $\square$

Corollary 6.3 (Commutative case). *In the commutative case of proposition 6.1, transport takes the form:*

$$\begin{aligned} D_S(g, l) &= \frac{d\phi(l)}{dl} \circ D_T(\phi(g), \phi(l)) \circ \frac{d\psi(\phi(g))}{d\phi(g)} \\ \text{embed}_S(g) &= \text{embed}_T(\phi(g)) \circ \frac{d\psi(\phi(g))}{d\phi(g)} \\ \text{finalize}_S(l) &= \frac{d\phi(l)}{dl} \circ \text{finalize}_T(\phi(l)) \end{aligned}$$

Proof. The LGP-result follows directly from the homomorphism properties and the chain rule (See appendix A.3). The factorization result follows directly by substituting the factorization $D_T = \text{finalize}_T \circ \text{embed}_T$ and grouping the composition by associativity. $\square$

Extension to smooth manifolds. For simplicity, the General Local Gradient Property, the Commutative Local Gradient Property, and the two transport propositions were stated for finite-dimensional vector spaces. However, the same arguments extend directly to finite-dimensional smooth manifolds equipped with smooth monoid operations, provided all derivatives are interpreted as maps between tangent spaces and their pullbacks on cotangent spaces. The proofs are unchanged, since they use only associativity, smoothness, and the chain rule.

Cancellability is transported If T is left and right cancellable, then so is S : $a \odot_S b = \psi(\phi(a) \odot_T \phi(b))$, which specializes to $\psi(\phi(a) \odot_T \phi(b)^{-1})$ in the case where T is a group⁴.

Transport as existence proof, not implementation The transport theorems above (Prop. 6.1, 6.2, 6.3) should be read as existence results and as a derivation aid: they guarantee that a finalize-embed factorization exists for a monoid S whenever an injective, smooth homomorphism into another monoid T with a finalize-embed factorization is available, and they supply a first candidate formula by direct substitution. They are not a claim that the substituted formula is the numerically preferable one to implement. In particular, the transported formulas route every quantity through $\phi(g)$ and $\phi(l)$ explicitly (e.g. $\exp(g_z)$ in the LWS -case), which can reintroduce exactly the numerical instabilities that motivate the execution-carrier construction (5). In practice, we treat the transported factorization as a specification to be refined: one is free to carry additional information through the accumulator A itself (rather than collapsing it inside embed). The factorization example in 4 illustrates this: the accumulator is a stable LWS -monoid, rather than the unstable, purely additive, version implied from direct transport.

⁴Note, however, that invertibility is not transported: $\psi(\phi(x)^{-1})$ is not defined if $\phi(x)^{-1}$ is not in the image of $\phi(S)$. Take for example LSE : $\phi(x) = \exp(x)$ and $\phi(x)^{-1} = -\exp(x)$, which can not be represented in the LSE monoid.

6.1 $GL_N(\mathbb{R})$ satisfies the General LGP

Given $a, b \in GL_N(\mathbb{R})$ with $a \odot b := ab$,

$$\begin{aligned} D_l(a \odot b, a)[\Delta] &= \Delta (a^{-1} (a \odot b))^{\top} \\ D_r(a \odot b, b)[\Delta] &= ((a \odot b) b^{-1})^{\top} \Delta \end{aligned}$$

which gives that

$$D(g, p, l)[\Delta] = (pl^{-1})^{\top} \Delta (p^{-1}g)^{\top} \quad (5)$$

with factorization through the additive matrix semigroup $(M_N(\mathbb{R}), +)$:

$$\begin{aligned} \text{finalize}_{GL_N(\mathbb{R})}(p, l)[e] &= l^{-\top} p^{\top} e p^{-\top} \\ \text{embed}_{GL_N(\mathbb{R})}(g)[\Delta] &= \Delta g^{\top} \end{aligned}$$

By the transport proposition, any monoid $(S, \odot) \hookrightarrow GL_N(\mathbb{R})$ admits a transported General LGP formulation using the following concrete instances of finalize and embed:⁵

$$\text{finalize}_S(p, l)[e] = \frac{d\phi(l)}{dl} [\phi(l)^{-\top} \phi(p)^{\top} e \phi(p)^{-\top}] \quad (6)$$

$$\text{embed}_S(g)[\Delta] = \frac{d\psi(\phi(g))}{d\phi(g)} [\Delta] \phi(g)^{\top} \quad (7)$$

6.2 $(\mathbb{R}^N, +)$ satisfies the Commutative LGP

A simpler and quite common special case of $GL_N(\mathbb{R})$ is $(\mathbb{R}^N, +)$. We show that this satisfies the Commutative LGP.

$$\begin{aligned} a \odot b &:= a + b \\ D_{\mathbb{R}^N}(a \odot b, a) &= \frac{da + b}{da} \\ &= \text{id} \end{aligned}$$

with factorization through the additive vector semigroup $(\mathbb{R}^N, +)$:

$$\begin{aligned} \text{finalize}_{(\mathbb{R}^N, +)}(l) &= \text{id} \\ \text{embed}_{(\mathbb{R}^N, +)}(g) &= \text{id} \end{aligned}$$

⁵Often, the image $\phi(S)$ is considerably restricted, as in the case of affine transforms with scalar scaling. While it embeds into $GL_{D+1}(\mathbb{R})$, it can be represented by $D+1$ parameters, rather than the $(D+1)^2$ of the full $GL_{D+1}(\mathbb{R})$. As such, the transport should be seen as a proof of existence, with computation being performed in a more performant and/or numerically stable representation. This extends to the gradient accumulator object A .

By transport, any monoid $(S, \odot) \hookrightarrow (\mathbb{R}^N, +)$ satisfies the commutative LGP with the following concrete instances of finalize and embed:

$$\text{finalize}_S(l) = \frac{d\phi(l)}{dl} \quad (8)$$

$$\text{embed}_S(g) = \frac{d\psi(\phi(g))}{d\phi(g)} \quad (9)$$

Example: LWS is transportable to $(\mathbb{R}^N, +)$ LogWeightedSum is transportable to $(\mathbb{R}^N, +)$ by transforming the log-space weight to the corresponding total weight by exponentiation, and by transforming the mean to the total sum by multiplication by the total weight, along with the inverse operation:

$$\begin{aligned} \phi(a) &= \{w : \exp(a_z), s : w \cdot a_\mu\} \\ \psi(x) &= \left\{ z : \log(x_w), \mu : \frac{1}{x_w} \cdot x_s \right\} \end{aligned}$$

The induced finalize-embed factorization is the unstable factorization presented in the example in section 4.

6.3 $(\mathbb{R}, \max)$ satisfies a Commutative LGP-style rule

A slightly more involved case is max, using tie-counting subgradient convention, i.e. $S := \{m : \mathbb{R}, t : \mathbb{N}_{>0}\}$ with $S^* \cong \mathbb{R}$, since cotangents are taken only with respect to the real-valued max output m ; the tie count t is discrete auxiliary state and receives no cotangent⁶. The tie-counting max operation is defined as

$$a \odot b = \begin{cases} \{m : a_m, t : a_t + b_t\} & \text{if } a_m = b_m \\ a & \text{if } a_m > b_m \\ b & \text{if } b_m > a_m \end{cases}$$

where

$$D(g, l)[\Delta] = \begin{cases} \frac{l_t}{g_t} \cdot \Delta & \text{if } g_m = l_m \\ 0 & \text{if } g_m > l_m \end{cases} \quad (10)$$

Factoring through the following partial semigroup⁷ $(A, \oplus)$, embed, and finalize:

⁶Strictly speaking, this uses a chosen subgradient convention rather than the smooth proposition as stated.

⁷Note that the operation $\oplus$ is defined only for compatible ordered pairs with $a_m \leq b_m$; while not total in general, this always hold for the sequences produced by max-scans.

$$\begin{aligned}
A &:= \{m : \mathbb{R}, \gamma : \mathbb{R}\} \\
a \oplus b &= \begin{cases} \{m : a_m, \gamma : a_\gamma + b_\gamma\} & \text{if } a_m = b_m \\ a & \text{if } a_m < b_m \end{cases} \\
\text{embed}(g)[\Delta] &= \left\{ m : g_m, \gamma : \frac{1}{g_t} \Delta \right\} \\
\text{finalize}(l)[e] &= \begin{cases} l_t e_\gamma & \text{if } l_m = e_m \\ 0 & \text{if } l_m < e_m \end{cases}
\end{aligned}$$

This generalizes to $\mathbb{R}^N$ with pointwise max and corresponding pointwise A .

Remark: Non-Cancellative Monoids and Sliding-Window Kernels While many standard sequence models rely on group-embeddable transitions (such as SSMS), the finalize-embed factorization of the LGP is not restricted to cancellative algebras, as shown by the max-example above. Another example is sliding-window causal operation (such as a causal 1D convolution or local attention) of window size K . This can be modeled as a scan over the non-cancellative *Right- K monoid* $(S, \odot)$, where:

$$a \odot b := \text{Right}_K(a \parallel b)$$

and the state space S consists of sequences of length at most K .

The backward pass of a Right- K scan factors through the Left- K monoid $(A, \oplus)$ acting as the accumulator semigroup, where:

$$u \oplus v := \text{Left}_K(u \parallel v)$$

The embedding map $\text{embed}(g)[\Delta] := [\Delta]$ yields a single-element sequence, and a parallel suffix scan over the Left- K monoid accumulates exactly the local window of K future cotangents:

$$a_i = [\Delta_i, \Delta_{i+1}, \dots, \Delta_{i+K-1}]$$

Finally, $\text{finalize}(p_i, x_i)$ uses local slicing derivatives to route this window of accumulated future cotangents back to the input x_i .

7 Complete Sketch

Given

8 Implementation sketches

What follows are implementation sketches that utilize the machinery we've built so far. For ease of presentation, the sketches are written for semantic monoids S , rather than execution carriers E . However, the corresponding implementation using execution carriers is fairly immediate.

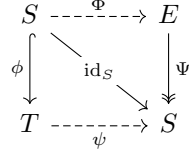


Figure 1: The algebraic relationships between the execution, semantic, and transport representations. The execution carrier E provides a numerically stable execution path (e.g., using the max-trick) via a surjective homomorphism $\Psi : E \twoheadrightarrow S$ with right inverse Φ . Differentiability properties are inherited by the semantic target S from T via transport, using an injective, smooth homomorphism $\phi : S \hookrightarrow T$ with smooth left inverse ψ .

8.1 Efficient Folds

For efficient monoidal folds we present a tiled, parallel, sketch of the forward pass using a fused $\text{map_fold} : X_{\text{tile}} \rightarrow S_{\text{summary}}$ function which computes local monoidal summaries S_{summary} from local input tiles X_{tile} , corresponding to the following non-tiled function:

$$\text{map_fold}(x)_i = \bigodot_j l_{i,j} \quad (11)$$

where $l_{i,j} = \text{map}(x_{i,j})$ is a monoidal factor corresponding to one input interaction. In the following pseudocode, bold symbols like $\mathbf{g}$, $\mathbf{p}$, $\mathbf{l}$, and $\mathbf{x}$ denote tiled quantities, with $\mathbf{x}_{b,r}$ being the tile at index (b,r) , while the non-bold symbols above denote non-tiled quantities, with $x_{i,j}$ denoting the interaction at index (i,j) . The parallel, tiled, forward pass takes the following form:

```

def FoldForwardPass( $x \in X$ )  $\rightarrow S_{\text{summary}}^B$  :
   $\mathbf{l}_{b,r} : S_{\text{summary}} \leftarrow \text{map\_fold}(\text{tile}(x)_{b,r})$ 
  if Commutative: // Parallel or fused kernel reduction.
     $\mathbf{g}_b : S_{\text{summary}} \leftarrow \bigodot_r \mathbf{l}_{b,r}$ 
  if General: // Parallel exclusive prefix scan.
     $\mathbf{p}_{b,r} : S_{\text{summary}} \leftarrow \bigodot_{t < r} \mathbf{l}_{b,t}$ 
    optionally store  $\mathbf{p}_{b,r}$  for backward pass
     $\mathbf{g}_b \leftarrow \mathbf{p}_{b,R_{\text{last}}} \odot \mathbf{l}_{b,R_{\text{last}}}$ 
  return  $\mathbf{g}_b$ 

```

For the backward pass, we use fused backward functions

$$\text{fused_bwd} : X_{\text{tile}} [\times S_{\text{summary}}] \rightarrow A_{\text{summary}} \rightarrow X_{\text{tile}}^*$$

that take an input tile (and exclusive prefix summary tile in the general case), a gradient accumulation summary A_{summary} , and produces local gradients X_{tile}^* , corresponding to the following non-tiled function:

$$\text{fused_bwd}(x, p^{\leftarrow})[\eta] = \sum_{i,j} \frac{d l_{i,j}}{d x} [\text{finalize}(p_{i,j}, l_{i,j})[\eta_i]] \quad (12)$$

$$\text{fused_bwd}(x)[\eta] = \sum_{i,j} \frac{d l_{i,j}}{d x} [\text{finalize}(l_{i,j})[\eta_i]] \quad (13)$$

where $p_{i,j}$ corresponds to the tile-local view of a global running prefix, computed using the left-boundary $p^{\leftarrow}$ and l :

$$p_{i,j} = p_i^{\leftarrow} \odot \left(\bigodot_{t \leq j} l_{i,t} \right) \quad (14)$$

The parallel, tiled, backward pass takes the following form

```
def FoldBackwardPass( $x \in X$ ,  $\mathbf{g} \in S_{\text{summary}}^B$ ,  $\Delta \in S_{\text{summary}}^{B*}$ )  $\rightarrow X^*$ :
   $\mathbf{x}_{b,r} : X_{\text{tile}} \leftarrow \text{tile}(x)_{b,r}$ 
   $\mathbf{a}_b : A_{\text{summary}} \leftarrow \text{embed}(\mathbf{g}_b)[\Delta_b]$ 
  if Commutative:
     $\delta_{b,r} : X_{\text{tile}}^* \leftarrow \text{fused\_bwd}(\mathbf{x}_{b,r})[\mathbf{a}_b]$ 
  if General:
    load or recompute  $\mathbf{p}_{b,r}$ 
     $\delta_{b,r} : X_{\text{tile}}^* \leftarrow \text{fused\_bwd}(\mathbf{x}_{b,r}, \mathbf{p}_{b,r})[\mathbf{a}_b]$ 
  // Parallel sum-reduce (tiling adjoint).
  return  $\text{tile}^*[\delta_{b,r}]$ 
```

8.2 Efficient Scans

For efficient monoidal scans we present a tiled, parallel, sketch of the forward pass using a fused function $\text{map_scan} : X_{\text{tile}} \rightarrow S_{\text{tile}}$ which is the scan analogue of map_fold : it computes a tile-local monoidal scan over a full intermediate tile S_{tile} from local input tiles X_{tile} , corresponding to the following non-tiled function:

$$\text{map_scan}(x)_{i,j} = \bigodot_{t \leq j} l_{i,t} \quad (15)$$

where $l_{i,j} = \text{map}(x_{i,j})$ is a monoidal factor corresponding to one input interaction.

```
def ScanForwardPass( $x \in X$ )  $\rightarrow S_{\text{tile}}^{B \times R}$ :
   $\mathbf{h}_{b,r} : S_{\text{tile}} \leftarrow \text{map\_scan}(\text{tile}(x)_{b,r})$ 
  // Parallel exclusive prefix scan
   $\mathbf{p}_{b,r} : S_{\text{summary}} \leftarrow \bigodot_{t < r} \mathbf{h}_{b,t_{\text{last}}}$ 
  store  $\mathbf{p}_{b,r}$  for backward pass
  return  $\mathbf{p}_{b,r} \odot \mathbf{h}_{b,r}$ 
```

For the backward pass, we use fused backward functions $\text{fused_bwd} : X_{\text{tile}} [\times S_{\text{tile}}] \rightarrow$

$A_{\text{tile}} \rightarrow X_{\text{tile}}^*$ that take an input tile (and prefix scan tile in the general case), a global suffix scanned gradient accumulation tile E_{tile} , and produce local gradients X_{tile}^* , corresponding to the following non-tiled function:

$$\text{fused_bwd}(x, y)[\eta] = \sum_{i,j} \frac{d l_{i,j}}{d x} [\text{finalize}(y_{i,j}, l_{i,j})[\eta_{i,j}]] \quad (16)$$

$$\text{fused_bwd}(x)[\eta] = \sum_{i,j} \frac{d l_{i,j}}{d x} [\text{finalize}(l_{i,j})[\eta_{i,j}]] \quad (17)$$

as well as a function $\text{embed_scan} : S_{\text{tile}} \rightarrow S_{\text{tile}}^* \rightarrow A_{\text{tile}}$, that take a global prefix scanned tile S_{tile} , a global gradient tile S_{tile}^* and produce tile-local suffix scanned gradient accumulation tile A_{tile} :

$$\text{embed_scan}(y)[\Delta]_{i,j} = \bigoplus_{t \geq j} \text{embed}(y_{i,t})[\Delta_{i,t}] \quad (18)$$

Backward scan:

```
def ScanBackwardPass( $x \in X$ ,  $y \in S_{\text{tile}}^{B \times R}$ ,  $\Delta \in S_{\text{tile}}^{B \times R^*}$ )  $\rightarrow X^*$ :
   $\mathbf{x}_{b,r} : X_{\text{tile}} \leftarrow \text{tile}(x)_{b,r}$ 
   $\eta_{b,r} : A_{\text{tile}} \leftarrow \text{embed\_scan}(\mathbf{y}_{b,r})[\Delta_{b,r}]$ 
   $\eta_{b,r}^{\rightarrow} : A_{\text{summary}} \leftarrow \bigoplus_{t \geq r} \eta_{b,t} \text{first}$  // Parallel suffix scan
   $\eta_{b,r}^{\Sigma} : A_{\text{tile}} \leftarrow \mathbf{a}_{b,r} \oplus \mathbf{a}_{b,r}^{\rightarrow}$ 
  if Commutative:
     $\delta_{b,r} : X_{\text{tile}}^* \leftarrow \text{fused\_bwd}(\mathbf{x}_{b,r})[\mathbf{a}_{b,r}^{\Sigma}]$ 
  if General:
     $\delta_{b,r} : X_{\text{tile}}^* \leftarrow \text{fused\_bwd}(\mathbf{x}_{b,r}, \mathbf{y}_{b,r})[\mathbf{a}_{b,r}^{\Sigma}]$ 
  // Parallel sum-reduce (tiling adjoint).
  return  $\text{tile}^*[\delta_{b,r}]$ 
```

Comparison with Blelloch-Style Scan Backpropagation The scan constructions above should be compared to the usual reverse-mode treatment of a Blelloch scan. In a standard parallel backward pass, one differentiates the scan circuit itself, which amounts to scanning backward over transfer operators describing how cotangents propagate from later prefixes to earlier inputs. The LGP view instead rewrites the backward pass into three stages: a local embedding of each upstream cotangent, a suffix scan in an auxiliary gradient accumulator, and a local finalization step. The asymptotic consequence is therefore not primarily a change in total work, but a simplification of the object scanned in the reverse pass.

For the literal additive scan on $(\mathbb{R}^N, +)$, there is no asymptotic gain: the backward pass is already just a suffix sum of N -vectors, so both the usual Blelloch backpropagation and the LGP formulation require $O(JN)$ work and $O(N \log J)$ span. In this case, the two viewpoints are essentially the same algorithm.

The $GL_N(\mathbb{R})$ case is different. A generic Blelloch-style backward scan propagates affine linear operators on $N \times N$ cotangents, so the scan combines objects using matrix multiplication, giving $O(JN^3)$ work and $O(N^3 \log J)$ span in the dense case. By contrast, the LGP factorization rewrites the backward pass as a suffix scan over ordinary $N \times N$ matrices under addition, with the matrix multiplications moved into the local embed/finalize steps. The total work remains $O(JN^3)$, but the span becomes $O(N^3 + N^2 \log J)$. Thus, for GL_N , the gain is not in total work but in reducing the cost of the scan tree itself.

More generally, the transport propositions show that the same distinction persists for monoids that are not themselves additive or matrix-valued, but admit injective, smooth homomorphisms into these representation spaces. If S transports into T with gradient accumulator A , the reverse scan can be performed over A , rather than a $S \times S$ dimensional Jacobian.

The same distinction appears in communication complexity. A parallel suffix scan over P processors requires $\Theta(\log P)$ communication rounds regardless of whether it is derived from Blelloch backpropagation or from the LGP. The difference is in the communicated state: communication volume is proportional to the size of the scan element. Thus, transport into $(\mathbb{R}^M, +)$ communicates $\Theta(M \log P)$ words per processor, while transport into $GL_M(\mathbb{R})$ communicates $\Theta(M^2 \log P)$ words per processor. These match specialized Blelloch-style backward scans for literal addition or matrix-valued states, but can be asymptotically smaller than a generic reverse scan in local coordinates on S .

Essentially, this finalize-embed factorization gives a principled way to derive specialized backward scans for Blelloch-style parallel prefix computations by identifying a simpler algebra (the gradient accumulator A) in which gradient accumulation can be scanned.

8.3 Efficient Scan-Fold

While scans do not see much memory savings, we can combine folds and scans, such that the intermediate state produced by the scan is reduced for each timestep by a commutative fold, keeping only the final folded value. Essentially fusing a Scan over R that produces $S^{B \times R}$ states with a commutative fold over B , producing final output F^R . This enables scans over wide states that are reduced before being pushed downstream, generalizing current Mamba style layers.

For this, we use a fused, tile-local, function that maps input interactions $x_{i,j}$ and scan state $p_{i,j}$ to local fold-factors $l_{i,j}$, which are folded over b :

$$\begin{aligned} \text{map_fold_inner} : X_{\text{tile}} \times S_{\text{tile}} &\rightarrow F_{\text{summary}} \\ \text{map_fold_inner}(x, p)_j &= \bigodot_i^F l_{i,j} \end{aligned}$$

where $l_{i,j} : F = \text{map}_{X \times S \rightarrow F}(x_{i,j}, p_{i,j})$ is a local fold factor constructed from an input interaction $x_{i,j}$ and corresponding scan state $p_{i,j}$.

```

def ScanFoldForwardPass( $x \in X$ )  $\rightarrow F_{\text{summary}}^R$  :
   $\mathbf{x}_{b,r} : X_{\text{tile}} \leftarrow \text{tile}(x)_{b,r}$ 
   $\mathbf{h}_{b,r} : S_{\text{tile}} \leftarrow \text{map\_scan}(\mathbf{x}_{b,r})$ 
  // Parallel exclusive prefix scan
   $\mathbf{p}_{b,r} : S_{\text{summary}} \leftarrow \odot_{t < r}^S \mathbf{h}_{b,t_{\text{last}}}$ 
  optionally store  $\mathbf{p}_{b,r}$  for backward pass
   $\mathbf{p}_{b,r}^{\text{tile}} : S_{\text{tile}} \leftarrow \mathbf{p}_{b,r} \odot_S \mathbf{h}_{b,r}$ 
  // Fused local fold over (inner) B
   $\mathbf{l}_{b,r} : F_{\text{summary}} \leftarrow \text{map\_fold\_inner}(\mathbf{x}_{b,r}, \mathbf{p}_{b,r}^{\text{tile}})$ 
  // Parallel reduction over (global) B
   $\mathbf{y}_r : F_{\text{summary}} \leftarrow \odot_b^F \mathbf{l}_{b,r}$ 
  return  $\mathbf{y}_r$ 

```

For the backward pass, we use the LGP for both the associative scan monoid S and the commutative fold F . For the commutative fold we define a fused backward mapping, analogous to `fused_bwd` used for efficient folds, that computes the gradient w.r.t. input values x and scan state p :

$$\text{fused_inner_bwd}(x, p)[e] = \sum_{i,j} \frac{\text{d} l_{i,j}}{\text{d}(x, p)} [\text{finalize}_F(l_{i,j})[e_j]]$$

With such a function, the scan-fold backward pass can be defined as follows:

```

def ScanFoldBackwardPass(
    x ∈ X,
    y ∈ FRsummary,
    Δ ∈ FRsummary*
) → X*:

    erF : AF,summary ← embedF(yr)[Δr]
    load pb,r
    xb,r : Xtile ← tile(x)b,r

    // 1. Reconstruct states (local)
    pb,rtile : Stile ← pb,r ⊙S map_scan(xb,r)

    // 2. Differentiate the Fold (local)
    (δxb,rfold : Xtile*, Δpb,rtile : Stile*) ← fused_inner_bwd(xb,r, pb,rtile)[erF]

    // 3. Differentiate the Scan
    // 3.1 Embed the cotangents (local)
    eb,r : Atile ← embed_scan(pb,rtile)[Δpb,rtile]

    // 3.2 Parallel (exclusive) suffix scan (global)
    eb,r→ : Asummary ← ⊕t>r eb,tfirst
    eb,rΣ : Atile ← eb,r ⊕ eb,r→

    // 4. Finalize (local)
    if Commutative:
        δb,rscan ← fused_bwd(xb,r)[eb,rΣ]
    if General:
        δb,rscan ← fused_bwd(xb,r, pb,rtile)[eb,rΣ]
    δb,r ← δb,rscan + δb,rfold
    // 5. Total Gradient (global)
    // Parallel global sum-reduce (tiling adjoint)
    return tile*[δb,r]

```

9 Instances

9.1 Attention

For single-headed attention, we use the *LWS*-monoid:

$$\text{attention}(Q, K, V)_i = \left(\bigodot_j \{z : \langle Q_i, K_j \rangle, \mu : V_j\} \right)_\mu \quad (19)$$

9.2 Single-distribution information theoretic measures

For a distribution p , parameterized as unnormalized logits, we can reuse the same LWS -monoid, but with scalar μ :

$$\text{stat}(p) = \bigodot_i \{z : p_i, \mu : p_i\} \quad (20)$$

For $g = \text{stat}(p)$,

$$\begin{aligned} g_z &= \ln \sum_i \exp(p_i) \\ g_\mu &= \sum_i \exp(p_i - g_z) \cdot p_i \\ &= \sum_i \bar{p}_i \cdot p_i \\ H(p) &= - \sum_i \bar{p}_i \ln \bar{p}_i \\ &= g_z - g_\mu \end{aligned}$$

where $\bar{p}_i = \exp(p_i - g_z)$ denotes the global probability corresponding to logit p_i .

9.3 Information Theoretic Measures on Two Distributions

For two distributions p and q , parameterized as unnormalized logits, we use what is essentially the product of two LWS -monoids:

$$\left\{ \begin{array}{l} \mathbf{p} : LWS(\{\mu : \mathbb{R}, \sigma : \mathbb{R}\}) \\ \mathbf{q} : LWS(\{\mu : \mathbb{R}, \sigma : \mathbb{R}\}) \end{array} \right\} \quad (21)$$

Then, for

$$\text{stat}(p, q) = \bigodot_i \{ \mathbf{p} : \{z : p_i, \mu : p_i, \sigma : q_i\}, \mathbf{q} : \{z : q_i, \mu : q_i, \sigma : p_i\} \} \quad (22)$$

we can compute the following, given $g = \text{stat}(p, q)$

$$\begin{aligned} \text{KL}(p||q) &= \sum_i \bar{p}_i \cdot (\ln \bar{p}_i - \ln \bar{q}_i) \\ &= \sum_i \bar{p}_i \cdot (p_i - g_{\mathbf{p}_z} - q_i + g_{\mathbf{q}_z}) \\ &= g_{\mathbf{p}_\mu} - g_{\mathbf{p}_z} - g_{\mathbf{p}_\sigma} + g_{\mathbf{q}_z} \\ \text{CE}(p||q) &= H(p) + \text{KL}(p||q) \\ &= g_{\mathbf{q}_z} - g_{\mathbf{p}_\sigma} \\ \text{Jeffreys}(p, q) &= g_{\mathbf{p}_\mu} - g_{\mathbf{p}_\sigma} + g_{\mathbf{q}_\mu} - g_{\mathbf{q}_\sigma} \end{aligned}$$

Which can be shrunk by removing unused terms, i.e. when computing cross-entropy, we only need $\{\mathbf{p} : LWS(\sigma), \mathbf{q} : LSE\}$, where LSE is the LogSumExp monoid.

Since the terms $\mathbf{p}$ and $\mathbf{q}$ are independent during the fold.

9.4 One-hot Cross-Entropy

For a distribution p , parameterized as unnormalized logits, and q a one-hot distribution $q_i = \delta_{it}$, we have

$$\text{CE}(q||p) = -\ln \bar{p}_t = \ln \left(\sum_i \exp(p_i) \right) - p_t$$

As such, the carrier needs only the log-partition of p and the logit p_t :

$$\begin{aligned} S &= \{z : LSE, \ell : \mathbb{R}\} \\ a \odot b &= \{z : a_z \odot_{LSE} b_z, \ell : a_\ell + b_\ell\} \end{aligned}$$

Then with:

$$\text{stat}(p, t) = \bigodot_i \{z : p_i, \ell : \delta_{it} p_i\} \quad (23)$$

we have that $\text{CE}(t||p) = g_z - g_\ell$ for $g = \text{stat}(p, t)$.

9.5 Affine transforms

An affine transform $f(x) = f_a \cdot x + f_b$ is a noncommutative monoid under composition and, as commonly used in sequence models, reverse composition: $g(f(x)) = (g \circ f)(x) = g_a(f_a \cdot x + f_b) + g_b$, where $f \odot g = g \circ f$. This gives the carrier $\{a : \mathbb{R}, b : \mathbb{R}^D\}$, where $(f \odot g) = \{a : g_a f_a, b : g_a \cdot f_b + g_b\}$, which transports to $GL_{D+1}(\mathbb{R})$.

The direct, additive, finalize-embed factorization then takes the form:

$$\begin{aligned} \text{embed}(g)[\Delta] &= \{a : g_a \Delta_a + \langle g_b, \Delta_b \rangle, b : \Delta_b\} \\ \text{finalize}(p, l)[\eta] &= \frac{1}{l_a} \cdot \{a : \eta_a - \langle p_b, \eta_b \rangle, b : p_a \cdot \eta_b\} \end{aligned}$$

Example: Monoid discovery by inverse transport Proposition 6.1 is not only a method to prove that a given monoid S satisfies the LGP, it can also be used in the other direction: Given T , we can define a new monoid S by specifying ϕ and ψ . As an example, we can construct an Affine function with log-space scaling and LWS -style addition:

$$\begin{aligned} \phi(a) &= \{a : \exp(a_\alpha), b : \phi_{LWS}(a_\beta)\} \\ \psi(x) &= \{\alpha : \log(x_a), \beta : \psi_{LWS}(x_b)\} \end{aligned}$$

The binary operation for this monoid is then:

$$\begin{aligned}
a \odot b &= \psi(\phi(a) \odot_{Affine} \phi(b)) \\
&= \left\{ \begin{array}{l} \alpha : \log(\exp(a_\alpha) \cdot \exp(b_\alpha)) \\ \beta : \psi_{LWS}(\exp(b_\alpha) \cdot \phi_{LWS}(a_\beta) + \phi_{LWS}(b_\beta)) \end{array} \right\} \\
&= \left\{ \begin{array}{l} \alpha : a_\alpha + b_\alpha \\ \beta : (b_\alpha \star a_\beta) \odot_{LWS} b_\beta \end{array} \right\} \\
&\text{where } s \star a = \{z : a_z + s, \mu : a_\mu\}
\end{aligned}$$

The result is a monoid where the scaling factor a is in logspace, and acts on an LWS monoid by *reweighting* the payload, rather than rescaling it. A consequence of this is that, like in attention, the output LWS payloads are convex combinations of the input LWS payloads. If we also, as is common in SSMS, restrict alpha to be non-positive, we get an incredibly stable, self-normalizing, scan-layer with neither vanishing nor exploding gradients. See appendix A.5 for a stable finalize-embed factorization for this monoid.

10 Discussion and Future Work

The algebraic results in this paper are neither deep nor complex. They are essentially immediate from commutativity, associativity, homomorphisms, and standard calculus. However, we believe that the main contribution lies in identifying the algebraic machinery that enables existing and future memory-efficient layers. All the better that that machinery is fairly straightforward!

A promising extension of this framework is applying the Scan-Fold primitive (Section 8.3) to non-cancellative, structural monoids to algebraically capture local sliding-window operations. For instance, a causal convolution or local attention mechanism of window size K can be modeled as a scan over the *Right- K monoid* (extracting the active window) fused with a commutative fold over the window elements (evaluating the attention weights).

Algebraically, the backward pass of this operation factors through a *Left- K parallel suffix scan* that evaluates the gradient via a data-independent anti-diagonal sum. Because the Right- K operation is purely structural, this finalization is completely independent of the primal activations l and p . Implementing this as an optimized, automatic compiler pass (e.g., in Triton or XLA)—utilizing overlapping tiling to execute the Scan-Fold in local SRAM without physically replicating the $O(L \cdot K \cdot D)$ intermediate states—is an exciting direction for future systems research.

References

- Bruun, L. M., Larsen, U. S., Hinnervskov, N. H., and Oancea, C. E. (2023). Reverse-mode ad of multi-reduce and scan in futhark. In *Proceedings of the*

- 35th Symposium on Implementation and Application of Functional Languages*, pages 1–14.
- Charlier, B., Feydy, J., Glaunès, J. A., Collin, F.-D., and Durif, G. (2021). Kernel operations on the gpu, with autodiff, without memory overflows. *Journal of Machine Learning Research*, 22(74):1–6.
- Chen, T., Xu, B., Zhang, C., and Guestrin, C. (2016). Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*.
- Dao, T. (2023). Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Re, C. (2022). Flashattention: Fast and memory-efficient exact attention with IO-awareness. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K., editors, *Advances in Neural Information Processing Systems*.
- Dean, J. and Ghemawat, S. (2008). Mapreduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113.
- Elliott, C. (2018). The simple essence of automatic differentiation. *Proc. ACM Program. Lang.*, 2(ICFP).
- Griewank, A. and Walther, A. (2000). Algorithm 799: revolve: an implementation of checkpointing for the reverse or adjoint mode of computational differentiation. *ACM Trans. Math. Softw.*, 26(1):19–45.
- Gu, A. and Dao, T. (2024). Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*.
- Guo, H., Yang, S., Goel, T., Xing, E. P., Dao, T., and Kim, Y. (2025). Log-linear attention. *arXiv preprint arXiv:2506.04761*.
- Hsu, P.-L., Dai, Y., Kothapalli, V., Song, Q., Tang, S., Zhu, S., Shimizu, S., Sahni, S., Ning, H., and Chen, Y. (2024). Liger kernel: Efficient triton kernels for llm training. *arXiv preprint arXiv:2410.10989*.
- Rush, K., Charles, Z., Garrett, Z., Augenstein, S., and Mitchell, N. (2024). Drjax: Scalable and differentiable mapreduce primitives in jax. *arXiv preprint arXiv:2403.07128*.
- Schenck, R., Rønning, O., Henriksen, T., and Oancea, C. E. (2022). Ad for an array language with nested parallelism. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE.
- Wijmans, E., Huval, B., Hertzberg, A., Koltun, V., and Krähenbühl, P. (2024). Cut your losses in large-vocabulary language models. *arXiv preprint arXiv:2411.09009*.

Yang, S., Kautz, J., and Hatamizadeh, A. (2025). Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations*, volume 2025, pages 29687–29707.

A Appendix

A.1 Proof for the general LGP (proposition 3.1)

Proof. Since $(T, \odot)$ is a monoid, l is a suffix of p , and p a prefix of g , there exists $a, b \in T$ such that $p = a \odot l$ and $g = a \odot l \odot b$.

Plugging this into eq. 1, we get $\frac{dg}{dl} = \frac{da \odot l \odot b}{dl} = D(a \odot l \odot b, a \odot l, l) = D(g, p, l)$. $\square$

A.2 Proof for the commutative LGP (corollary 3.2)

Proof. For any partition of H into H^1, H^2 , with corresponding partial products $p_1 := \bigodot_i H_i^1$ and $p_2 := \bigodot_i H_i^2$ and global product $g := \bigodot_i H_i$

$$\begin{aligned} g &= p_1 \odot p_2 = p_2 \odot p_1 && \text{(By commutativity and associativity)} \\ \frac{dg}{dp_1} &= \frac{dp_1 \odot p_2}{dp_1} \\ &= D(p_1 \odot p_2, p_1) && \text{(By the condition } \frac{da \odot b}{da} = D(a \odot b, a)) \\ &= D(g, p_1) \end{aligned}$$

This extends trivially to $p_1 := h_i$, i.e., $H^1 = [h_i]$:

$$\begin{aligned} \frac{dg}{dh_i} &= \frac{dp_1}{dh_i} \circ D(g, p_1) \\ &= D(g, p_1) && \left(\frac{dp_1}{dh_i} = \text{id} \right) \\ &= D(g, h_i) \end{aligned}$$

$\square$

A.3 Proof for general transport (proposition 6.1)

Proof. Due to ϕ being a homomorphism with inverse ψ on its image:

$$\begin{aligned} \phi(s_1 \odot s_2) &= \phi(s_1) \otimes \phi(s_2) \\ s_1 \odot s_2 &= \psi(\phi(s_1) \otimes \phi(s_2)) \end{aligned}$$

Let

$$\begin{aligned} g_S &= s_1 \odot s_{1+1} \odot \cdots \odot s_N \\ p_S &= s_1 \odot s_{1+1} \odot \cdots \odot s_n \\ l_S &= s_m \odot s_{m+1} \odot \cdots \odot s_n \end{aligned}$$

for $1 \leq m \leq n \leq N$. Let $t_i = \phi(s_i)$, and

$$\begin{aligned} g_T &= \phi(g_S) = t_1 \otimes t_{1+1} \otimes \cdots \otimes t_N \\ p_T &= \phi(p_S) = t_1 \otimes t_{1+1} \otimes \cdots \otimes t_n \\ l_T &= \phi(l_S) = t_m \otimes t_{m+1} \otimes \cdots \otimes t_n \end{aligned}$$

The chain rule, in combination with the homomorphism properties and the fact that $(T, \otimes)$ satisfies the LGP, gives that

$$\begin{aligned} \frac{dg_S}{dl_S} &= \frac{dl_T}{dl_S} \circ \frac{dg_T}{dl_T} \circ \frac{dg_S}{dg_T} \\ &= \frac{dl_T}{dl_S} \circ D_T(g_T, p_T, l_T) \circ \frac{dg_S}{dg_T} \\ &= \frac{d\phi(l_S)}{dl_S} \circ D_T(\phi(g_S), \phi(p_S), \phi(l_S)) \circ \frac{d\psi(\phi(g_S))}{d\phi(g_S)} \\ &= D_S(g_S, p_S, l_S) \end{aligned}$$

□

A.4 Proof for commutative transport (corollary 6.3)

Proof. Let $g_S = s_1 \odot s_{1+1} \odot \cdots \odot s_N$, and let l_S be the product over an arbitrary index set $I \subseteq \{1, \dots, N\}$: $l_S := \bigodot_{i \in I} s_i$. Since ϕ is a homomorphism, $g_T = \phi(g_S)$, $l_T = \phi(l_S)$.

The chain rule, in combination with the homomorphism properties and the fact that $(T, \otimes)$ satisfies the commutative LGP, gives that

$$\begin{aligned} \frac{dg_S}{dl_S} &= \frac{dl_T}{dl_S} \circ \frac{dg_T}{dl_T} \circ \frac{dg_S}{dg_T} \\ &= \frac{dl_T}{dl_S} \circ D_T(g_T, l_T) \circ \frac{dg_S}{dg_T} \\ &= \frac{d\phi(l_S)}{dl_S} \circ D_T(\phi(g_S), \phi(l_S)) \circ \frac{d\psi(\phi(g_S))}{d\phi(g_S)} \\ &= D_S(g_S, l_S) \end{aligned}$$

□

A.5 Finalize-embed factorization of the Affine LWS example

The stable finalize-embed factorization for the monoid discussed in section 9.5 follows much the same pattern as that for the LWS monoid (4).

The factorization given by direct transport (to the affine case) is:

$$\begin{aligned} \text{embed}(g)[\Delta] &= \left\{ \begin{array}{l} a : \Delta_\alpha + \Delta_{\beta_z} \\ b : \exp(-g_{\beta_z}) \cdot \left\{ \begin{array}{l} w : \Delta_{\beta_z} - \langle \Delta_{\beta_\mu}, g_{\beta_\mu} \rangle \\ s : \Delta_{\beta_\mu} \end{array} \right\} \end{array} \right\} \\ \text{finalize}(p, l)[\eta] &= \left\{ \begin{array}{l} \alpha : \eta_a - \exp(p_{\beta_z}) \cdot (\eta_{b_w} + \langle \eta_{b_s}, p_{\beta_\mu} \rangle) \\ \beta : \exp(p_\alpha - l_\alpha + l_{\beta_z}) \cdot \left\{ \begin{array}{l} z : \eta_{b_w} + \langle \eta_{b_s}, l_{\beta_\mu} \rangle \\ \mu : \eta_{b_s} \end{array} \right\} \end{array} \right\} \end{aligned}$$

Which due to the raw exponents is unstable, it can, however, be refined to a stable carrier by substituting b with an LWS -carrier:

$$A := \{\tau : \mathbb{R}, e : LWS(\{w : \mathbb{R}, s : \mathbb{R}^D\})\} \quad (24)$$

$$a \odot_A b = \{\tau : a_\tau + b_\tau, e : a_e \odot_{LWS} b_e\} \quad (25)$$

$$\text{embed}(g)[\Delta] = \left\{ \begin{array}{l} \tau : \Delta_\alpha + \Delta_{\beta_z} \\ e : \left\{ \begin{array}{l} z : -g_{\beta_z} \\ w : \Delta_{\beta_z} - \langle \Delta_{\beta_\mu}, g_{\beta_\mu} \rangle \\ s : \Delta_{\beta_\mu} \end{array} \right\} \end{array} \right\} \quad (26)$$

$$\text{finalize}(p, l)[\eta] = \left\{ \begin{array}{l} \alpha : \eta_\tau - \lambda \cdot (e_w + \langle e_s, p_{\beta_\mu} \rangle) \\ \beta : \gamma \cdot \left\{ \begin{array}{l} z : e_w + \langle e_s, l_{\beta_\mu} \rangle \\ \mu : e_s \end{array} \right\} \end{array} \right\} \quad (27)$$

$$e = \eta_e$$

$$\text{where } \lambda = \exp(p_{\beta_z} + e_z) \quad (28)$$

$$\gamma = \exp(p_\alpha - l_\alpha + l_{\beta_z} + e_z)$$

In the case where the observed output is restricted to the internal LWS payload (β_μ), Δ_α and Δ_{β_z} are both zero, giving the simplified formulation:

$$\begin{aligned}
A &:= LWS(\{w : \mathbb{R}, s : \mathbb{R}^D\}) \\
\text{embed}(g)[\Delta] &= \left\{ \begin{array}{l} z : -g_{\beta_z} \\ w : -\langle \Delta_{\beta_\mu}, g_{\beta_\mu} \rangle \\ s : \Delta_{\beta_\mu} \end{array} \right\} \\
\text{finalize}(p, l)[\eta] &= \left\{ \begin{array}{l} \alpha : -\lambda \cdot (\eta_w + \langle \eta_s, p_{\beta_\mu} \rangle) \\ \beta : \gamma \cdot \left\{ \begin{array}{l} z : \eta_w + \langle \eta_s, l_{\beta_\mu} \rangle \\ \mu : \eta_s \end{array} \right\} \end{array} \right\} \\
\text{where } \lambda &= \exp(p_{\beta_z} + \eta_z) \\
\gamma &= \exp(p_\alpha - l_\alpha + l_{\beta_z} + \eta_z)
\end{aligned}$$

While the terms λ and γ might look numerically frightening, η_z